Assignment 1: OCR in the Cloud

Due Date: 22/11

Submit your questions to Gal

Abstract

In this assignment you will code a real-world application to distributively OCR algorithms on images, upload the extracted texts to S3, and generate an output HTML file.

More Details

The application is composed of a local application and instances running on the Amazon cloud. The application will get as an input a text file containing a list of URLs of images. Then, instances will be launched in AWS (workers). Each worker will download image files, use some OCR library to identify texts in those images (if any) and display the image with the text in a webpage.

The use-case is as follows:

1. User starts the application and supplies as input a file with URLs of image files, an integer n stating how many files a single worker can process, and an optional argument terminate, if received the local application sends a terminate message to the Manager.
2. User gets back an output html file with images and their texts.

Input and Output Files

The input file is a list of URLs, one URL per line.

The output is an html file, containing the result images with links to the original images.

A sample input file can be found here.
A sample output file can be found here.

If an exception occurs while performing an operation on an image, or the image file is not available, then output line for this file will be: <a link to the input file > <a short description of the exception>.

System Architecture

The system is composed of 3 elements:

- Local application
- Manager
- Workers

The elements communicate with each other using queues (SQS) and storage (S3). It is up to you to decide how many queues to use and how to split the jobs among the workers, but, and you will be graded accordingly, your system should strive to work in **parallel**. It should be as **efficient** as possible in terms of time and money, and **scalable**.

Local Application

The application resides on a local (non-cloud) machine. Once started, it reads the input file from the user, and:

- Checks if a Manager node is *available* on the EC2 cloud. If it is not, the application will start a new manager node.
  An available Manager is an active Manager which currently serving less than *manager-clients-n* clients (as defined by the *manager-clients-n* parameters).
- Uploads the file to S3.
- Sends a message to an SQS queue, stating the location of the file on S3
- Checks an SQS queue for a message indicating the process is done and the response (the summary file) is available on S3.
- Gets the summary output file from S3.
- Creates an html output file representing the results.
- In case of *terminate* mode (as defined by the command-line argument), sends a termination message to the Manager.

IMPORTANT: There might be more than one than one local application running at the same time.

The Manager

A manager process resides on an EC2 node. It checks a special SQS queue for messages from local applications. Once it receives a message it:

- If the message is that of a new task it:
  - Downloads the input file from S3.
  - Creates an SQS message for each URL in the input file.
  - Checks the SQS message count and starts Worker processes (nodes) accordingly.
    - The manager should create a worker for every *worker-files-n* files, if there are no running workers.
    - If there are *k* active workers, and the new job requires *m* workers, then the manager should create m-k new workers, if possible.
      For any case don't run more than 19 instances – AWS uses to block students who try this.
    - Note that while the manager creates a node for each *n* messages, it does not delegate messages to specific nodes. All worker nodes take their messages from the same SQS queue; it might be the case that with *2n* messages, hence two worker nodes, one node processed *n+(n/2)* messages, while the other processed only *n/2*.

  - If the message is a termination message, then the manager:
    - Does not accept any more input files from local applications.
    - Waits for all the workers to finish their job, and then terminates them.
    - Creates response messages for the jobs, if needed.
    - Terminates.

IMPORTANT: the manager must process requests from local applications simultaneously; meaning, it must not handle each request at a time, but rather work on all requests in parallel.

The Workers

A worker process resides on an EC2 node. Its life cycle is as follows:

Repeatedly:

- Gets a message from an SQS queue.
- Downloads the image indicated in the message.
- Performs OCR on the image.
- Notify the manager of the text associated with that image.
- Remove the processed message from the SQS queue.

IMPORTANT:

- If an exception occurs, the worker should recover from it, send a message to the manager of the input message that caused the exception together with a short description of the exception, and continue working on the next message.
- If a worker stops working unexpectedly before finishing its work on a message, some other worker should be able to handle that message.

The Queues and Messages

As described above, SQS queues are used for:

- communication between the local application and the manager.
- communication between the manager and the workers.

Specifically, we will have the following messages:

- *new task* message from the application to the manager (location of an input file)
- *new image task* message from the manager to the workers (URL of a specific image)
- *done OCR* message from a worker to the manager (URL of image and identified text)
- *done* message from the manager to the application (S3 location of the output file)

It is up to you to decide how many queues you want (you can have different queues for different tasks, or one queue, whatever you find most convenient). Be ready to explain your choices.

Running the Application

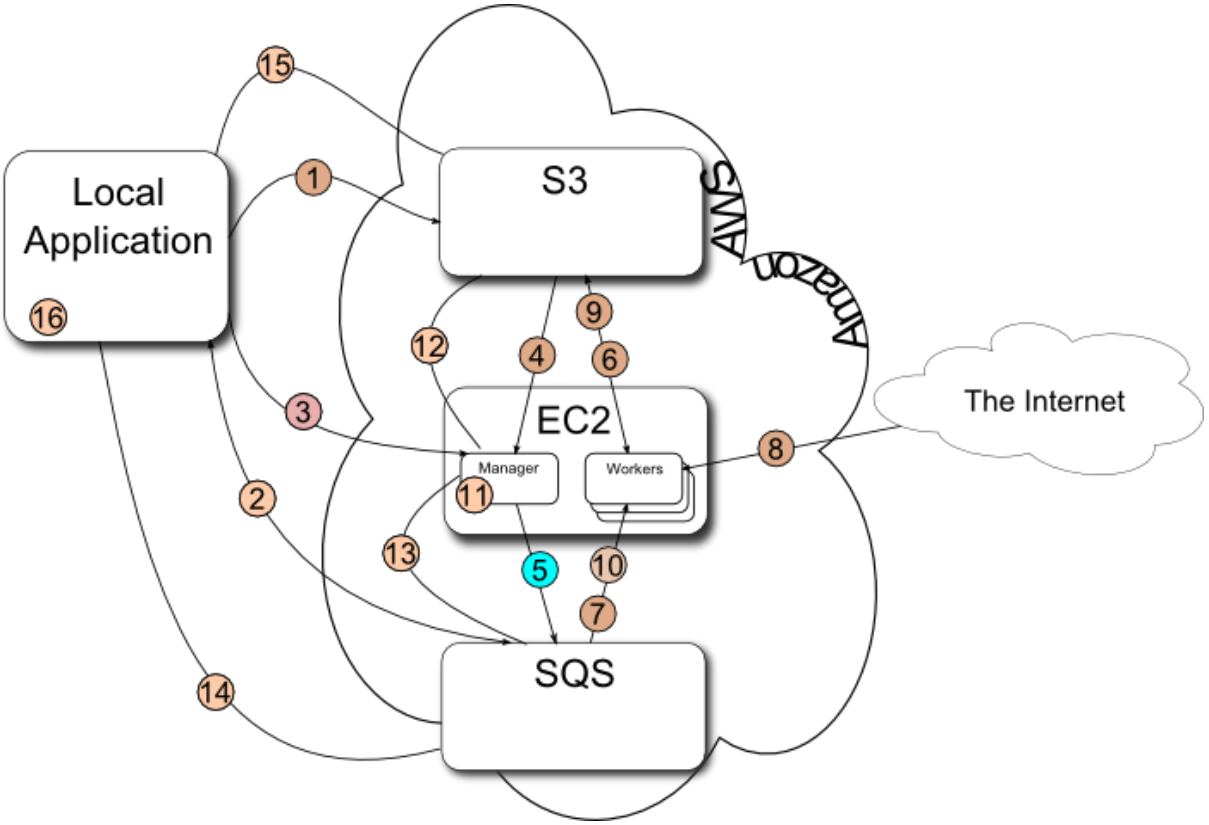
The application should be run as follows:

```
>java -jar yourjar.jar inputFileName outputFileName manager-clients-n worker-files-n [terminate]
```

where:

- *yourjar.jar* is the name of the jar file containing your code.
- *inputFileName* is the name of the input file.
- *outputFileName* is the name of the output html file.
- *manager-clients-n* is the maximum clients per manager
- *worker-files-n* is the workers' files ratio (max files per worker).
- *terminate* indicates that the application should terminate the manager at the end.

System Summary



1. Local Application uploads the file with the list of image urls to S3.
2. Local Application sends a message (queue) stating the location of the input file on S3.
3. Local Application does one of the two:
   - Starts the manager.
   - Checks if a manager is available and if not, starts a new one.
4. Manager downloads the input from S3 (a list of image urls)
5. Manager creates an SQS message for each URL from the input list.
6. Manager bootstraps nodes to process messages.
7. Worker gets a message from an SQS queue.
8. Worker downloads the image indicated in the message.
9. Worker performs OCR on the image.
10. Worker puts a message in an SQS queue indicating the original URL of the image and the extracted text.
11. Manager reads all Workers' messages from SQS and creates one summary file per client, once all client's URLs in the input file have been processed.
12. Manager uploads the summary file to S3.
13. Manager posts an SQS message about the summary file.
14. Local Application reads SQS message.
15. Local Application downloads the summary file from S3.
16. Local Application creates html output file.
17. Local application send a terminate message to the manager if it received *terminate* as one of its arguments.

Getting Started

- Read the assignment description.
- Read the reading the material, and make sure you understand it.

- Create a maven project.
- Write the code apply OCR on image file (based on the given OCR tool, as follows), run it on your computer and make sure it works.
- Write the local application code and make sure it works.
- Write the manager code, run it on your computer and make sure it works.
- Run the manager, the local application, together with a worker on your computer and make sure they work.
- Run the local application on your computer, and let it start and run the manager on EC2, and let the manager start and run the workers on EC2.

Technical Stuff

- Getting started with AWS:
    - Read the guide in the AWS academy lab - all you need to know is presented there, and is updated to the latest features and libraries of AWS.
    - Make sure that you place the .pem (credentials) file in the right location, and where your AWS CLI expects it to be -- this is up to you to make sure you do it correctly. Otherwise, you won't be able to communicate with the AWS services.
    - Be aware of AWS regions - make sure that the time of the region you're using (the region of the instances) is consistent with your system (the region of your aws console, the region of the AWS dashboard). Since the session key of AWS academy is only valid for several hours, large time differences can mean that while your access keys are indeed 'fresh', it is not recognized by AWS.
      This link may be useful.

- AMI Image

A virtual machine (VM) is a software implementation of a machine (i.e. a computer) that executes programs like a physical machine. In our case: EC2 instance. An image of a virtual machine is (in simple words) a copy of the VM, which may contain an OS, data files, and applications (just like your personal computer) - in our case: Amazon Machine image (AMI).

You need an AMI composed of any data needed for running the Manager and the Workers (basically, Linux, Java JDK, Amazon AWS cli, the jar file of the Manager and the Worker classes, and the OCR jar).

You can create a new AMI from the Amazon EC2 Instances view, or from the console of an EC2 instance:

Downloading from EC2 console: In Linux, the command *wget* is usually installed. You can use it to download web files from the shell.
Example:
wget https://www.guteiageimagnberg.org/files/1659/1659-0.txt -O a.txt
will download the content at https://www.gutenberg.org/files/1659/1659-0.txt and save it to a file named a.txt. wget man

Installing from EC2 console: In Ubuntu (or Debian) Linux, you can use the apt-get command (assuming you have root access to the machine). Example: apt-get install *wget* will install the *wget* command if it is not installed. You can use it to install Java, or other packages. apt-get man.

In case AWS disable the option for creating new AMI, use the user-data script for downloading the required jars each time the node is initialized.

- The AWS SDK

The assignment will be written in Java, you'll need the SDK for Java for it.

AWS Maven dependencies:

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>ec2</artifactId>
  <version>2.57.0</version>
</dependency>

<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>s3</artifactId>
  <version>2.57.0</version>
</dependency>

<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>sqs</artifactId>
  <version>2.57.0</version>
</dependency>
```

- SQS Visibility Time-out

Read the following regarding SQS timeout, understand it, and use it in your implementation:
Visibility Timeout

- Bootstrapping

When you create and boot up an EC2 node, it turns on with a specified image, and that's it. You need to load it with software and run the software in order to do something useful. We refer to this as "bootstrapping" the machine.
As taught in class, the bootstrapping process of a new node is based on user data scripts, and another guide. User-data allows you to pass a shell-script to the machine, to be run when it starts up. Notice that the script you're passing should be encoded to base64.
Your user-data scripts can be written in any language you want (e.g. Python, Perl, tsch, bash). bash is a very common choice. Your scripts are going to be very simple. Nonetheless, you might find these bash tutorials useful.

- Checking if a Manager Node is Active

You can check if the manager is running by listing all your running instances, and checking if one of them is a manager. Use the "tags" feature of Amazon EC2 API to mark a specific instance as one running a Manager: using tags, CreateTagsRequest API

- Third-party libraries

To convert the given images, you can use one of the following OCR tools (listed with their maven dependencies, to be added to the Worker pom.xml), or any other available tool you prefer:

Tesseract

```
<dependency>
  <groupId>net.sourceforge.tess4j</groupId>
  <artifactId>tess4j</artifactId>
  <version>5.20.0</version>
</dependency>
```

javaOCR

```
<!-- https://mvnrepository.com/artifact/net.sourceforge.javaocr/javaocr-core -->
<dependency>
  <groupId>net.sourceforge.javaocr</groupId>
  <artifactId>javaocr-core</artifactId>
 <version>1.0</version>
</dependency>
```

Asprise

```
<dependency>
  <groupId>com.asprise.ocr</groupId>
  <artifactId>java-ocr-api</artifactId>
  <version>15.3.0.3</version>
</dependency>
```

Grading

- The assignment will be graded in a frontal setting.
- All information mentioned in the assignment description, or learnt in class is mandatory for the assignment.
- You will be reduced points for not reading the relevant reading material, not implementing the recommendations mentioned there, and not understanding them.
- Students belonging to the same group will not necessarily receive the same grade.
- All the requirements in the assignment will be checked, any missing functionality will cause a point reduction. Any additional functionality will compensate for lost points. You have the "freedom" to choose how to implement things that are not precisely defined in the assignment.

Notes

Cloud services are cheap but not free. Even if they were free, waste is bad. Therefore, please keep in mind that:

- It should be possible for you to easily remove all the things you put on S3. You can do that by putting them in a specified bucket under a folder, which you could delete later.

- While it is the Manager's job to turn off all the Worker nodes, do verify yourself that all the nodes really did shut down, and turn of the manger manually if it is still running.
- You won't be able to download the files unless you make them public.
- You may assume there will not be any race conditions; conditions were 2 local applications are trying to start a manger at the same time etc.

A Very Important Note about Security

As part of your application, you will have a machine on the cloud contacting an amazon service (SQS and S3). For the communication to be successful, the machine will have to supply the service with a security credentials (password) to authenticate. Security credentials is sensitive data – if someone gets it, they can use it to use amazon services for free (from your budget). You need to take good care to store the credentials in a secure manner. One way of doing that is by compressing the jar files with a password.

Submission

Submit a zip file whichcontains:

- all sources and class files, without the libraries that you're supposed to download;
- the output of running your system on the sample file.
- a text file called README with instructions on how to run your project, and an explanation of how your program works. It will help you remember your implementation. Your README must also contain what type of instance you used (ami + type:micro/small/large…), how much time it took your program to finish working on the input files, and what was the n you used.

Mandatory Requirements

- Be sure to submit a README file. Does it contain all the requested information? If you miss any part, you will lose points. Yes including your names and ids.
- Did you think for more than 2 minutes about security? Do not send your credentials in plain text!
- Did you think about scalability? Will your program work properly when 1 million clients connected at the same time? How about 2 million? 1 billion? Scalability is very important aspect of the system, be sure it is scalable!
- What about persistence? What if a node dies? What if a node stalls for a while? Have you taken care of all possible outcomes in the system? Think of more possible issues that might arise from failures. What did you do to solve it? What about broken communications? Be sure to handle all fail-cases!
- Threads in your application, when is it a good idea? When is it bad? Invest time to think about threads in your application!
- Did you run more than one client at the same time? Be sure they work properly, and finish properly, and your results are correct.
- Do you understand how the system works? Do a full run using pen and paper, draw the different parts and the communication that happens between them.
- Did you manage the termination process? Be sure all is closed once requested!
- Did you take in mind the system limitations that we are using? Be sure to use it to its fullest!
- Are all your workers working hard? Or some are slacking? Why?
- Is your manager doing more work than he's supposed to? Have you made sure each part of your system has properly defined tasks? Did you mix their tasks? Don't!
- Lastly, are you sure you understand what distributed means? Is there anything in your system awaiting another?

All of this need to be explained properly and added to your README file. In addition to the requirements above.